

Checking Alethe Proofs with LEAN-SMT

A summary of the work on the `alethe` branch of LEAN-SMT

Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

Abstract. LEAN-SMT [2] reconstructs CVC5’s CPC proofs into Lean proofs, driving the solver through its API. This note describes an extension that checks textual Alethe proofs, as produced by CVC5 and VERiT and elaborated by CARCARA, against SMT-LIB problems, reusing LEAN-SMT’s term reconstruction, lemma libraries, and arithmetic decision procedures. We describe the pipeline, how Alethe’s contexts and clauses are handled, how the proof rules are reconstructed, and how the proof reaches the kernel: step by step, in batches, or as one term. We close with the state of the evaluation and what it revealed about the proofs themselves.

1 System Overview

Figure 1 depicts the pipeline. The input is an SMT-LIB problem and an Alethe proof of its unsatisfiability, produced by CVC5 (`--proof-format-mode=alethe`) or by VERiT. The proof first goes through CARCARA’s [3] *elaborator*, which turns the coarse rules of the solver output into steps the checker can replay: the `polyeq` pass removes the polyequalities and implicit contexts of `assume`, `refl` and `forall_inst`; the `local` pass canonicalizes `trans`, `cong` and resolution, writing the pivots as arguments; and the `core` pass reduces the rules we chose not to reconstruct in Lean to rules we do reconstruct, restricted to a list given on the command line. The elaborated proof is then read by LEAN-SMT. Its *parser* eliminates the binding constructs of the text, its *realizer* hands every term to CVC5’s own parser through the Lean API, so that the reconstruction of terms is the one LEAN-SMT already has for CPC proofs, and its *reconstructor* replays the proof rule by rule, sending each step to Lean’s kernel as it goes. The verdict distinguishes a *valid* proof, in which every step was reconstructed and accepted by the kernel, from a *holey* one, in which some steps had to be trusted; the trusted steps are reported by rule, and, when the checker runs as a tactic, become goals for the user.

The checker is available in three forms. The command `#check_alethe "P.smt2"` `"P.alethe"` checks a proof from files and prints the verdict with per-phase timings; the `alethe` tactic closes a Lean goal the way the `smt` tactic does, translating the goal, obtaining an Alethe proof from CVC5 (in process, or from an external binary), running CARCARA, and reconstructing the result as a proof term of the goal; and the underlying `checkAlethe` function serves an executable for batch evaluation. The trusted computing base is the one of CPC reconstruction: Lean’s kernel, and CVC5’s parser for the statement.

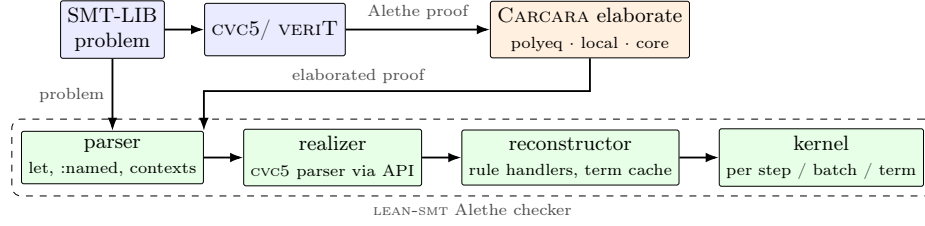


Fig. 1. Checking an Alethe proof. The solver’s proof is elaborated by CARCARA; the checker parses and realizes it with CVC5’s parser, replays it with LEAN-SMT’s reconstruction, and submits the steps to Lean’s kernel.

The rest of this section follows the pipeline: the treatment of the proof text (Section 1.1), the replay of steps and their submission to the kernel (Section 1.2), the reconstruction techniques per rule (Section 1.3), and the changes this required in CARCARA (Section 1.4). Section 2 reports the evaluation.

1.1 From Proof Text to Terms

Binding constructs. An Alethe proof, as CARCARA prints it, shares subterms with $(! \ t : \text{named } @p_n)$ annotations and, in VERiT’s case, with `let`. The parser eliminates both over a hash-consed arena of S-expressions, so that substitution never copies a term: a `let` is a scoped parallel binding, and a name stands for the annotated term from its definition onward. Anchor variables, the $(x \ S)$ and $(:= x \ t)$ arguments of an `anchor` command, are renamed to globally fresh symbols inside their block. The realizer then serializes each literal once, binding every shared subterm on first use as a fresh constant $s\tilde{n}$, so that CVC5’s hash-consing and LEAN-SMT’s term cache keep the Lean terms shared as well.

Contexts. Alethe’s subproofs under an anchor with $(:= x \ t)$ arguments carry a context Γ , and a step $\Gamma \triangleright s \simeq u$ inside it means $\text{subst}(\Gamma)(s) = u$: the substitution applies to the left-hand side only, and $\text{subst}(\Gamma, x \mapsto t) = \text{subst}(\Gamma) \circ \{x \mapsto t\}$, as in the semantics of Barbosa et al. for formula processing [1] and in CARCARA. The checker encodes an assignment as a `let`-bound local, so that inside the block the replaced variable unfolds to its value definitionally, and the bind rules read the substitution off that unfolding. To give the two sides of a judgment their two meanings, the parser carries two environments per context, the *substituted* one (with the renamings) and the *fixed* one (the enclosing environment plus the context’s own variables), and parses a unit clause $(= \ 1 \ r)$ with `1` under the first and `r` under the second. Where no substitution is in scope the two coincide. This is what makes VERiT’s nested renamings checkable, where an inner `bind` replaces a name that the enclosing `bind` had just introduced as a fixed variable; such proofs are valid under the paper’s calculus but outside the canonical fragment its algorithm produces, and were previously unrepresentable.

The same machinery absorbs a quirk of VERiT’s sharing: with `--proof-with-sharing`, VERiT names terms that are *open* in `let`-bound variables and reuses the names under other bindings of those variables. Read as SMT-LIB, where a name denotes the closed term at its definition, most of VERiT’s QF_UF proofs state

assumptions that are not the problem’s assertions. Read as macros over the let-bound variables they mention, they do. Both CARCARA and the checker now resolve a name anew in every scope it is used in, closed names being unaffected.

Realization. Terms are realized by CVC5’s parser in the higher-order logic `HO_ALL`, so that a `choice` binder can be represented as a per-sort constant applied to a `lambda` and later mapped to `Classical.epsilon`. Integer numerals in real-only logics are read as reals, as both solvers do. From this point on, the reconstruction of sorts and terms is LEAN-SMT’s `reconstructTerm`, unchanged.

1.2 Replaying Steps and Reaching the Kernel

Steps and clauses. A step is handed to the rule reconstructors as a record of its clause literals, rule, premises (with their literals and the Lean proof of their clause), arguments, discharged assumptions, and, for the closing step of a subproof, the anchor’s variables, assignments and last step. Reconstructors are registered with an attribute, `@[alethe_rule_reconstruct]`, one per theory, and dispatch on the rule name. A clause `(c1 l1 ... ln)` is the right-nested disjunction of its literals, definitionally `orN [l1, ..., ln]`, so that the `orN` lemmas of LEAN-SMT apply. Alethe’s clauses are sets, and elaborated resolutions state their conclusions modulo permutation and duplicate removal, so a computed clause is aligned with the stated one by `orN_of_map`: an index map from the stated literals to the computed ones, checked by the kernel by evaluation, proves the stated clause from the computed one; the associative-commutative rewriting of `ac_rfl` is kept only as a fallback.

Step by step. CPC reconstruction in LEAN-SMT builds one proof term for the whole proof and submits it to the kernel once. The Alethe checker instead checks every top-level step on its own: the step’s proof is type-checked by `Kernel.check` against its stated clause in a local context holding the problem’s symbols and assertions and only the earlier steps the proof mentions, as hypotheses; the proof term is then dropped and the step becomes a hypothesis for the steps after it. Steps inside a subproof are kept as terms, since the closing step’s proof abstracts over the block’s assumptions and variables, and are checked with it. Memory stays bounded by the largest step rather than the whole proof, a failure is attributed to the step the kernel rejected, and the checked count is exact. The cost is that every kernel call starts with an empty inference cache.

Batches, and one term. Between the two extremes, consecutive top-level steps can be gathered into one kernel call. A batch $e_1 : c_1, \dots, e_n : c_n$ becomes the term

$$(\lambda h_1 : c_1. (\lambda h_2 : c_2. \dots (\lambda h_n : c_n. \mathbf{trivial}) e_n \dots) e_2) e_1,$$

which the kernel accepts exactly when every e_i proves c_i under the hypotheses of the steps before it: the binders keep those hypotheses opaque, each conclusion stays an explicit annotation, and one call shares its cache over the batch. Each proof is abstracted once over the earlier hypotheses of its batch, so construction is linear in the batch. The batch is bounded by a number of steps or by the number of term nodes it holds (counted with sharing), and a rejected batch is re-

checked step by step, so that what is trusted is what the kernel actually rejected. Batches can also be dispatched to concurrent tasks, at most a fixed number in flight, joined in step order; the calls are independent by construction, since each establishes its conclusions from hypotheses rather than from the earlier proofs. Finally, a *term mode* mirrors CPC reconstruction, inlining every step into the steps that use it and submitting one closed theorem; it is what the `alethe` tactic uses, and the shape the evaluation compares against.

1.3 Reconstructing the Rules

The checker handles 93 Alethe rule names, in about 4 300 lines of Lean plus the generated RARE table, following the three techniques of CPC reconstruction: theorems, tactics, and verified programs. The dividing line with CARCARA is drawn per rule: a rule is reconstructed in Lean when a direct proof is cheap and robust, and reduced by CARCARA’s core pass otherwise, which the `--core-rules` flag we added lets the pipeline select rule by rule.

Via theorems. The clausal rules of Alethe, the CNF axioms (`and_pos`, `or_neg`, `implies_pos`, `equiv_neg1`, the `ite` and `xor` families) and the elimination rules (`and`, `not_or`, `implies`, `equiv2`, ...), are the CPC lemmas of LEAN-SMT applied with the indices CARCARA’s `local` pass writes as arguments. The connective definitions, the selection axioms `ite_then_intro` and `ite_else_intro` that the core pass derives `ite_intro` from, `la_disequality`, the division rules of CVC5 (`div_intro`, `div_by_zero_intro`) and the like are new lemmas in an Alethe-specific library of 46 theorems.

Via tactics. `la_generic` is Farkas’ lemma: the negated literals are introduced as hypotheses, integer bounds are tightened by the gcd of their coefficients as CARCARA does (a strict bound with unit gcd by `Int.add_one_le_iff` alone; the rounding cases by `omega`, given just the bound, the shape `poly_norm` establishes and the negated goal), the bounds are scaled by the rational coefficients given as arguments (integer bounds cast to rationals in mixed combinations), summed with LEAN-SMT’s `sum_ub` lemmas, normalized by `poly_norm`, and closed by the sign of the constant. Both details matter at scale: a `la_generic` with five hundred literals ran five hundred `omega` calls over the whole local context before, and now takes a few seconds. `cong` aligns the premises with the argument positions, in either orientation, and falls back to definitional equality for arguments that differ only by the context’s substitutions or by `Decidable` instances; congruence over n -ary `distinct`, whose reconstruction is a conjunction rather than an application, rewrites the arguments one at a time. `onpoint` is proved directly as a propositional extensionality: the eliminated binder may sit anywhere in the prefix and the guard may occur under further binders. The binder rules (`bind`, `sko_forall`, `sko_ex`, `let`) abstract the block’s variables with `forall_congr`, `exists_congr_eq` and the epsilon lemmas of LEAN-SMT; the clausal form of `bind` closes the quantified literal wherever it sits in the clause.

Via reflection. `poly_simp` and `poly_simp_rel` reuse LEAN-SMT’s verified polynomial normalizer, and `evaluate` the `decide` path. The normalizer proves an equation by `Eq.refl` on the normal forms of its two sides, so the kernel is what

computes them, and its cost on the thousand-variable sums of the SMPT benchmarks was the largest single source of failures in the evaluation. It folded the expression through a sorted insertion, copying the partial polynomial at every term of a left-associated sum, which is what SMT-LIB’s n -ary $+$ reconstructs to: a sum of 400 variables took the kernel 25 s and 8.9 GB, and a thousand exhausted the memory limit. It now does what CARCARA does, transposed to what the kernel can reduce: the monomials are collected in one pass into a flat list (carrying a sign, or a rational multiplier, so that subtraction and division by a constant cost nothing) and sorted once by a bottom-up merge sort whose merge adds like monomials. Everything stays structurally recursive, with the sort’s pass count a `Nat` bound, because the kernel does not unfold well-founded recursion; the denotation theorems hold for unsorted input too, so sortedness is needed only for completeness. The same sum of 400 variables now costs the kernel 1.3 s, and 800 cost 3.6 s at flat memory. For `aci_simp`, associativity, commutativity and idempotence of a connective layer, associative-commutative rewriting took seconds per step on VERiT proofs with two-hundred-conjunct layers, so we wrote a normalizer in the style of `poly_norm`:

```

inductive Tree | unit | leaf (i : Nat) | node (l r : Tree)
def bits : Tree → Nat
| unit => 0
| leaf i => Nat.shiftLeft 1 i
| node l r => Nat.lor (bits l) (bits r)
theorem denoteAnd_eq (ctx : Context) (a b : Tree) (m : Nat)
  (ha : bits a = m) (hb : bits b = m) :
  denoteAnd ctx a = denoteAnd ctx b

```

The top layer of a side is reified into a `Tree` whose leaves index a context of atoms, every subterm that is not the connective being an atom compared syntactically, as in CARCARA’s checker; its normal form is the bitset of its leaves, which the kernel computes in one structural pass with its accelerated `Nat.shiftLeft` and `Nat.lor`, and `denoteAnd_eq` turns two equal bitsets, both established by `Eq.refl`, into the equality of the formulas. Soundness is `denoteAnd ctx t ↔ ∀ i, (bits t).testBit i → ctx i`, proved by induction on the tree. A 200-atom layer costs the kernel 33 ms, against 34 s for a first attempt with a nested inductive and an insertion sort, where the kernel’s unfolding of nested structural recursion dominated. The same normalizer now serves CVC5’s `ACI_NORM` in CPC proofs.

Rewrite rules. CVC5’s rewrites reach Alethe as `rare_rewrite` steps naming a RARE rule and its arguments. A generator reads the RARE file CARCARA uses and produces, per rule, a Lean theorem statement and an entry of a name-keyed table; the proofs are written by hand between markers the generator preserves. 117 theorems cover the 107 rules the table lists, with `Int` and `Rat` instances chosen by the arguments’ sorts, list arguments spliced, and integer arguments cast where a rule mixes sorts; five statements of the multiplication-tangent family are stated but not yet proved, and seven n -ary rules have no statement, one of which, `distinct-false`, is handled by a dedicated proof from the reflexive disequality of the expanded conjunction. The `*_simplify` rules of Alethe are

not reconstructed at all: the core pass, restricted to them with `--core-rules`, rewrites each into a chain of RARE steps, which the table then covers.

Legacy rules. VERIT’s `ac_simp`, `qnt_cnf`, `ite_intro` and `bfun_elim` are likewise delegated to the core pass, which since its latest revision reduces `bfun_elim` completely, both the expansion of a quantifier over Boolean variables into its instances and the replacement of a Boolean argument of an application by an if-then-else, and derives `ite_intro` from the selection axioms. The application form of `bfun_elim` is also reconstructed natively, by case analysis on the Boolean arguments, for proofs elaborated without delegating it.

Instances. One consequence of substitution is worth recording. The instantiation of a quantified Boolean variable that occurs in an if-then-else condition carries the `Decidable` instance the quantified body was reconstructed with, `Classical.propDecidable` for an open condition, while the stated conclusion, reconstructed on its own, carries the structural instance for the now-closed condition. The two are propositionally but not definitionally equal, so the kernel rejects the step. A small procedure walks the two terms and closes such differences with `Subsingleton.elim`; `forall_inst` applies it when the instantiated body and the stated conclusion are not definitionally equal.

1.4 Changes in CARCARA

Besides being the elaborator, CARCARA is the reference the checker is measured against, and this work changed it in five places, all on the `bv-fixes` branch. The `--core-rules` flag restricts the core passes to the listed rules, so that a pipeline can delegate exactly the rules its consumer does not reconstruct. Under `--expand-let-bindings`, the `let` steps and their `bind_let` subproofs are trivialized, since a consumer expanding lets has nothing left to prove there. The `polyeq` pass canonicalizes Skolem witnesses, and the `sko_forall` and `sko_ex` rules become strict under elaborated granularity, which removed the last discrepancy in VERIT’s nested Skolemizations; `qnt_join` merges a whole prefix of nested quantifiers. Finally, named terms open in let-bound variables are macros under let expansion, as described in Section 1.1. The core transformation of `bfun_elim` and `ite_intro` was revised on the `coreAlethe` branch and merged.

2 Evaluation So Far

2.1 SMT-LIB

The corpus. The evaluation runs, per solver, over the unsatisfiable non-incremental SMT-LIB benchmarks of QF_UF, QF_LIA and QF_LRA (9 812 each, from the SMT-LIB 2025 catalog), on 16-core nodes with one core, 12 GB and 2 700 s per benchmark: 150 s for the solver, 300 s each for CARCARA’s check and the elaboration, and 900 s for each of the two LEAN-SMT arms, which share one solve and one elaboration so that the two shapes are compared on the same proof. Times below are CPU, not wall: the checker reads its build over a network filesystem, which costs several seconds of wall time on a cold node.

Table 1. The round, 9812 benchmarks per solver. *Valid* counts proofs checked with no trusted step; the two LEAN-SMT columns are the two kernel submission shapes.

solver	logic	pipeline			LEAN-SMT, per step		LEAN-SMT, one term	
		unsat	CARCARA	elab.	valid	timeout	valid	timeout
CVC5	QF_UF	4 324	4 297	4 294	2 526	1 762	1 952	1 770
	QF_LIA	2 794	2 509	2 509	2 272	190	1 460	163
	QF_LRA	581	526	526	416	110	199	91
	total	7 699	7 332	7 329	5 214	2 062	3 611	2 024
VERiT	QF_UF	4 193	4 174	4 174	2 444	1 728	2 048	1 579
	QF_LIA	2 607	2 309	2 308	2 058	171	995	444
	QF_LRA	584	388	388	278	108	93	102
	total	7 384	6 871	6 870	4 780	2 007	3 136	2 125

Outcomes. Table 1 gives the round. CARCARA accepts 95 to 99% of the QF_UF proofs and 82 to 90% of the arithmetic ones (what it rejects is almost entirely VERiT’s `lia_generic`, whose steps it does not check either); elaboration then succeeds on all but a handful. The checker validates 5 214 of the 7 329 elaborated CVC5 proofs and 4 780 of the 6 870 VERiT ones. The picture per logic is uneven in an informative way: the arithmetic proofs are small and the checker gets 91% (QF_LIA) and 79% (QF_LRA) of them for CVC5, against 59% of QF_UF, whose proofs are an order of magnitude larger.

Per step against one term. Figure 2 puts the two shapes side by side. Per-step checking dominates in every logic and for both solvers: it validates 1 603 proofs (CVC5) and 1 644 (VERiT) that the one-term shape does not, and there is not a single proof in the other direction. On the proofs both check it is also slightly cheaper. The one-term failures that are not timeouts split in two. About six hundred per solver are the single kernel call rejected for excessive memory, the cost of holding the whole derivation at once. The other thousand per solver were a defect of term mode found by this run: step by step, a premise is a hypothesis whose declared type is the clause the proof states, while in one term it is the proof itself, whose inferred type is only definitionally that clause, and the congruence reconstructor, reading a premise’s type, saw something else. A term-mode step now carries its stated type, and the one-term column of the next run will be correspondingly higher; the memory failures remain, and the measurement that led us to keep per-step checking as the default (Section 1.2) holds at corpus scale. It is the opposite of what the CPC path does.

Against CARCARA. Figure 4 compares the whole LEAN-SMT pipeline, elaboration included, with CARCARA’s own check of the solver’s proof. CARCARA checks a QF_UF proof in a median of 0.79s (CVC5) and 0.14s (VERiT), and elaborates it in 1.10s and 0.33s; the checker takes a median of 51s and 45s on the proofs it validates. The arithmetic proofs are far smaller: there CARCARA’s median is a hundredth of a second in QF_LIA and three tenths in QF_LRA, against 4s and 6s for the checker (CVC5). The whole round costs CARCARA 5.7 and 7.5 CPU hours against the checker’s 641 and 604, and the ratio grows with the proof: the

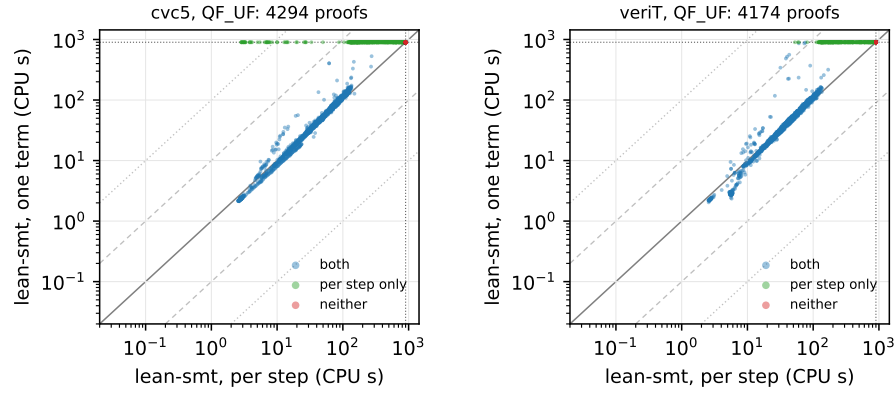


Fig. 2. The two kernel submission shapes on the same QF_UF proofs, CPU seconds. A point on a dotted border is a proof the corresponding shape did not check; the grey lines are equality and factors of ten and a hundred.

timeouts sit above proofs CARCARA disposes of in a second or two. That is the price of a proof term the kernel accepts, and it makes the interesting question not whether the checker competes with a dedicated one but how far up the size distribution it reaches.

How the cost accumulates. Figure 5 plots, for each configuration, the cumulative time against the number of benchmarks it validated, as the LEAN-SMT paper does. The shape is the familiar one: the checker follows CARCARA closely over the first few hundred benchmarks and then separates, both in slope and in where it stops. Reading the curves at a fixed budget rather than at the end gives the practical summary: within the 7.3 CPU hours CARCARA needs for every CVC5 proof, the checker validates roughly the first 1400 of them, and it needs ten times that budget to reach 2526.

What is left. Across the 6175 QF_UF tasks of both solvers the trusted steps number five: two **resolution** and one **subproof** the kernel rejected for memory on steps with thousands of literals, and three **subproof** type mismatches that were a real gap in the congruence over n -ary **distinct**, since fixed. Rule coverage for this logic is therefore complete, and what remains is scale: the timeouts start at 19000 elaborated steps for VERiT and 29000 for CVC5, with medians of 33000 and 52000. In the arithmetic logics, still running, the picture is the same with two additions: VERiT’s **lia_generic** is an oracle rule and trusted by design (163 proofs), and **la_generic** (105 VERiT proofs) and **poly_simp** (27 CVC5 proofs) steps over the thousand-variable sums of the SMPT Referendum benchmarks exhausted the kernel’s memory. Reproducing the latter on the same benchmark at ten sizes separated three causes, all since fixed: the polynomial normalizer’s insertion fold (Section 1.3), the per-literal tightening that gave **omega** the whole local context and ran it even for unit gains, and a **trans** reconstructor whose definitional fallback ran unconditionally because do-notation hoists a ($\leftarrow \dots$)

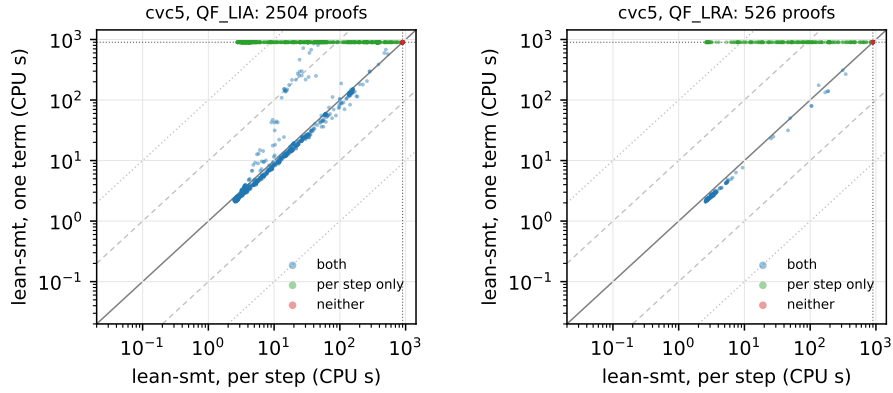


Fig. 3. The same comparison on `cvc5`’s arithmetic proofs, `QF_LIA` (left) and `QF_LRA` (right). The proofs are smaller, so both shapes reach further, and the one-term shape still loses 812 and 217 benchmarks.

out of an `else if`. The thousand-variable `la_generic` that exhausted 10 GB now checks in 7s and 2.6 GB, the 503-literal one in 24s. The two `la_generic` failures that were not resource limits needed the rounding of a non-strict integer bound to a multiple of its coefficients’ gcd, which the integer-incompleteness benchmarks exercise; that is fixed as well. The SMT-LIB round will be repeated once the Sledgehammer round now running has been analyzed.

2.2 The Sledgehammer Set

The corpus. The second round runs over the 5 000 SMT2 problems of the “Seventeen provers under the hammer” baseline set, the quantified Isabelle goals the LEAN-SMT paper evaluates on, with the same three arms and the same limits, and with `CVC5` given the instantiation options that paper used. These proofs are two to three orders of magnitude smaller than the SMT-LIB ones: a median of 61 elaborated steps for `CVC5` and 115 for `VERiT`, against tens of thousands in `QF_UF`.

Outcomes. Table 2 gives the round. Coverage is close to total: the checker validates 2 858 of the 2 867 elaborated `CVC5` proofs (99.7%) and 2 520 of the 2 595 `VERiT` ones (97.1%), with one and three timeouts. What limits the round is the solver, not the checker — 2 869 and 2 681 of the 5 000 problems are proved unsatisfiable at all — and, for `VERiT`, CARCARA’s own check, which rejects 83 proofs.

Per step against one term. On proofs this size the two shapes are nearly indistinguishable — 2 725 against 2 858 for `CVC5`, 2 508 against 2 520 for `VERiT`, at the same median cost — which is the other side of the SMT-LIB finding: the one-term shape loses only when the derivation is too large to hold at once. Of the one-term failures, 133 (`CVC5`) and 12 (`VERiT`) are the term-mode premise-type

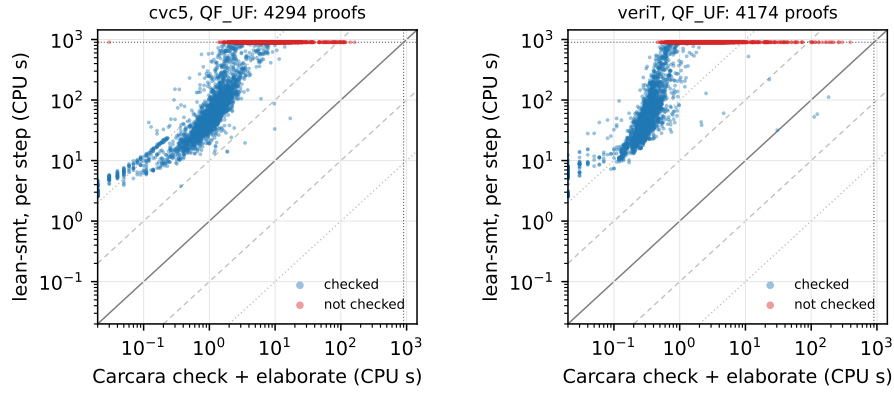


Fig. 4. CARCARA’s check plus the elaboration against LEAN-SMT’s per-step check, QF_UF, CPU seconds. Red points are proofs the checker did not finish within its limit.

Table 2. The Sledgehammer round, 5 000 problems per solver. The columns are those of Table 1.

solver	pipeline			LEAN-SMT, per step		LEAN-SMT, one term	
	unsat	CARCARA	elab.	valid	timeout	valid	timeout
CVC5	2 869	2 868	2 867	2 858	1	2 725	1
VERiT	2 681	2 598	2 595	2 520	3	2 508	3

defect this round exposed, since fixed; with it the two columns would essentially coincide, and every remaining difference is a proof the per-step arm also fails.

Cost. The proofs are small enough that the checker’s median of 5.0 s (CVC5) and 5.1 s (VERiT) is almost entirely the cost of loading the library, not of checking; the whole round costs it 4.7 and 5.8 CPU hours, against 72.2 and 92.8 hours of solving. CARCARA checks and elaborates in a median of 0.02 s and 0.01 s, for 0.04 and 0.02 CPU hours over the round. The few timeouts sit at the top of the size distribution — 516 elaborated steps for CVC5’s one, and 2 876 to 3 015 for VERiT’s three.

How the cost accumulates. Figure 8 is the cumulative plot of Figure 5 for this corpus, and it looks different in an instructive way. Restricted to the proofs each configuration validates, solving is cheap — a median of a third of a second for CVC5 and a tenth for VERiT — so the curves are dominated by what comes after it. CARCARA disposes of all 2 867 CVC5 proofs in 1.37 CPU hours including the solving, and of 2 595 VERiT ones in 1.22; the checker takes 5.82 and 4.97 for 2 858 and 2 520 of them, four times as much for all but a handful fewer. The checker’s curve is close to a straight line, because on proofs this small its cost is the fixed one of loading the library rather than anything about the derivation; that is

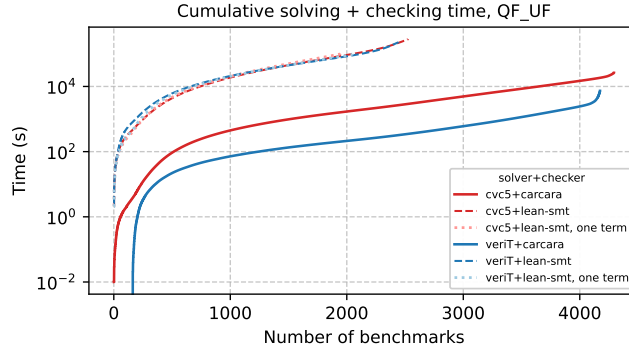


Fig. 5. Cumulative solving and checking time against the number of QF_UF benchmarks whose proof the configuration validated, in the style of the LEAN-SMT paper. Solving is included, which is why the CARCARA curves start where they do. CARCARA validates 4 294 CVC5 proofs in 7.3 CPU hours and 4 174 VERiT proofs in 2.0; the checker validates 2 526 and 2 444 in 78 and 64 hours, and the one-term curves stop earlier still.

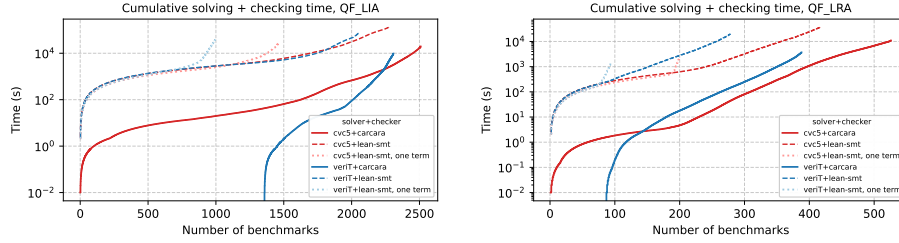


Fig. 6. Cumulative time in the arithmetic logics. CARCARA validates every QF_LIA proof of CVC5 in 5.4 CPU hours and the checker 2 272 of them in 36; in QF_LRA, 3.0 hours against 10 for 416 of 526.

also why the two submission shapes lie on top of each other. Within CARCARA’s whole budget the checker validates the first 1 059 (CVC5) and 963 (VERiT).

What is left. The residue is small and, for each solver, of one kind. CVC5 leaves nine proofs: eight whose `bind` step the kernel rejects with a type mismatch, and one genuine `hole`. VERiT leaves 51, of which 47 are `onepoint`: 37 because the anchor assigns two or three variables at once where the reconstructor supports one, and 13 because the guard equality sits in a body shape its traversal does not reach (three proofs hit both). The remaining four are three `qnt_simplify` steps with no reconstructor and one `symm/trans` pair. `onepoint` is therefore the single rule worth generalizing for this corpus, and CARCARA’s own rule fixes the specification: its `extract_points` traversal descends through negations, quantifiers, implications and disjunctions with a polarity, and admits every equality that forces a binder. CARCARA’s `core` pass also reduces the rule outright, which

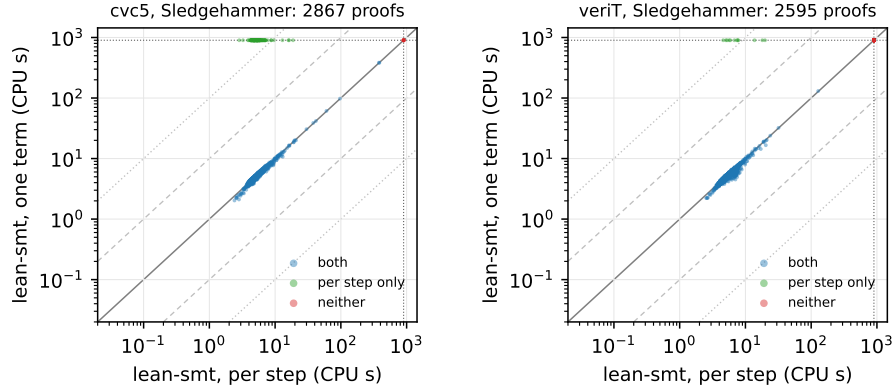


Fig. 7. The two submission shapes on the Sledgehammer proofs, CPU seconds. The band along the diagonal is the fixed cost of loading the library.

the checker can fall back on: delegating it costs about twice the steps on the affected subproof and removes the rule entirely.

What the proofs revealed. Checking against a second, independent reading of the rules paid off in both directions. On the producer side: VERiT’s open named terms, its identity substitutions ($:=$ ($?x$ S) $?x$) in every `bind`, its fresh names colliding with inner binders, and the eleven proofs whose `bfun_elim` the core pass silently kept, now logged and reduced. On the checker side: an `onepoint` guard refutation building `Or.inl` with an application builder that cannot infer the other disjunct, exposed only when the cluster reached the CVC5 UF benchmarks; the `Decidable` instance mismatch above; the quadratic construction of batch terms, caught by the first batch sweep; the term-mode premise types; and the hoisted definitional checks, which cost 49s on a `trans` step whose links all matched syntactically. Every finding is now a regression test.

References

1. Barbosa, H., Blanchette, J.C., Fleury, M., Fontaine, P.: Scalable fine-grained proofs for formula processing. *Journal of Automated Reasoning* 64(3), 485–510 (2020)
2. Mohamed, A., Mascarenhas, T., Khan, H., Barbosa, H., Reynolds, A., Qian, Y., Tinelli, C., Barrett, C.: Lean-SMT: An SMT tactic for discharging proof goals in Lean. In: *CAV 2025*
3. Andreotti, B., Lachnitt, H., Barbosa, H.: Carcara: An efficient proof checker and elaborator for SMT proofs in the Alethe format. In: *TACAS 2023*

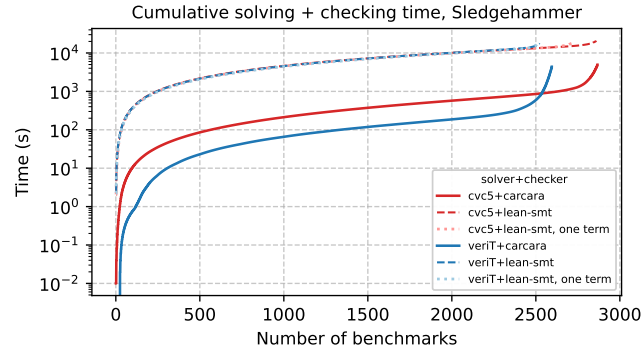


Fig.8. Cumulative solving and checking time against the number of Sledgehammer benchmarks whose proof the configuration validated. The LEAN-SMT curves are nearly straight, and the two submission shapes coincide.